PSI 2025-2026 – Objectif 1 : Graphe

Résumé

Ce document donne quelques instructions et conseils pour vous guider dans la réalisation de l'objectif 1, qui consiste à implémenter un type Graphe en C#, basé sur une matrice d'adjacence.

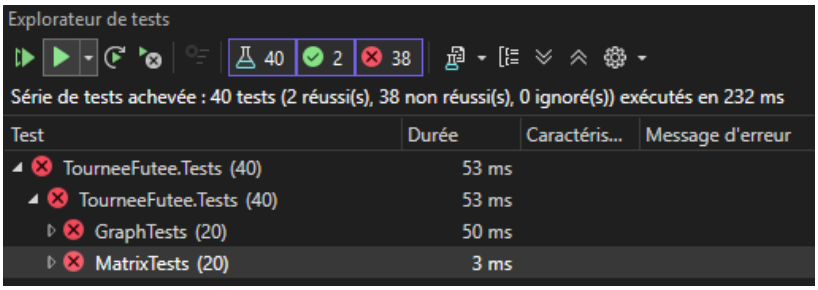
1 – Bootstrap

Récupération et ouverture de la solution fournie

Une solution Visual Studio vous est fournie pour démarrer le projet.

Un des membres du projet réalise les opérations suivantes :

1. Télécharger la solution zippée sur DVL, ressource « Bootstrap Visual Studio »
2. Dézipper la solution dans le répertoire du dépôt Git local utilisé pour PSI
3. Ouvrir la solution dans Visual Studio (**cliquez sur TourneeFutee.sln**)
4. Lancez les tests unitaires : clic droit sur le projet « TourneeFutee.Tests » puis « Exécuter les tests »
   ⇨ Le code doit compiler correctement, et les tests se lancer. En principe, vous devriez avoir 38 tests échoués (19 par campagne).



5. Si tout fonctionne correctement, soumettre les nouveaux fichiers sur le dépôt distant (*commit* + *push*)

Puis, **les autres** membres du groupe effectuent un *pull* pour récupérer les fichiers.

A présent, tous les membres disposent de la même solution dans Visual Studio. Elle contient deux projets :

- `TourneeFutee` : projet de type console, contenant le programme un début d'implémentation d'un graphe (classe `Graph`) et d'une matrice (classe `Matrix`)
- `TourneeFutee.Test` : projet MSTest, contenant deux campagnes de tests unitaires :
  - `GraphTests` : tests unitaires du type `Graph`
  - `MatrixTests` : tests unitaires du type `Matrix`

Modifications autorisées

Dans le projet `TourneeFutee.Test` : **aucune** modification n'est autorisée[1].

Dans le projet `TourneeFutee` :

- Modifications interdites :
  - Renommer les classes `Graph` ou `Matrix`
  - Modifier les signatures des méthodes imposées (nom, types) dans `Graph` et `Matrix`
  - Ajouter un setter dans une propriété en lecture seule.
- Modifications autorisées :
  - Définir le corps des méthodes imposées (y compris les getters) dans les classes `Graph` et `Matrix`
  - Ajouter des attributs ou des méthodes dans les classes `Graph` et `Matrix` (méthode utilitaire à usage interne = privée)
  - Ajouter des classes.

2 – Type matrice

Cette partie consiste à modéliser et implémenter un type matrice, qui sera utilisé pour stocker la matrice d'adjacence du graphe, mais aussi par l'algorithme de Little dans l'objectif 2.

Identification des collections génériques appropriées

- (Re)découvrez les collections génériques disponibles en C# dans la page de documentation dédiée.
- Sélectionnez un sous-ensemble de collections susceptibles d'être utilisées pour stocker une matrice.
- Pour chaque collection sélectionnée, étudiez la complexité temporelle pour les opérations suivantes :
  - Ajout/suppression d'une ligne/colonne en début/milieu/fin de matrice
  - Accès à la valeur d'une case par ses indices
- Identifiez la collection permettant de minimiser la complexité temporelle des **accès aux valeurs**. A complexité égale, privilégiez :
  - Celle qui minimise la complexité des autres opérations
  - Puis la facilité de mise en œuvre.

[1] Il est inutile d'essayer d'augmenter votre note en « trafiquant » les tests : les encadrants utiliseront la campagne de tests de référence pour vous évaluer.

Modélisation

Le type matrice en C# est modélisé par une classe imposée : `Matrix`.

- Identifiez les associations entre `Matrix` et les autres classes utilisées par `Matrix` (collections, exceptions, etc.).
- Représentez l'ensemble sous la forme d'un diagramme de classes UML.

Implémentation en C#

Complétez la classe `Matrix` fournie pour que chaque méthode ait le comportement décrit dans le commentaire, par exemple :

```
/* Insère une ligne à l'indice `i`. Décale les lignes suivantes vers le bas.
 * Toutes les cases de la nouvelle ligne contiennent DefaultValue.
 * Si `i` = NbRows, insère une ligne en fin de matrice
 * Lève une ArgumentOutOfRangeException si `i` est en dehors des indices valides
 */
2 références
public void AddRow(int i)
{
    // TODO : implémenter
}
```

La méthode doit effectuer les opérations décrites dans le commentaire, et lever d'éventuelles exceptions en cas d'erreur (comme c'est le cas ici).

Pour ce faire, vous devez écrire le corps des méthodes imposées (au niveau des commentaires `TODO : implémenter`), mais vous pouvez ajouter toutes les attributs et méthodes que vous jugerez pertinents.

Validez le comportement de chaque méthode en lançant le(s) test(s) unitaire(s) associé(s) dans la campagne de tests `MatrixTests`.

Par exemple, pour `AddRow()`, il y a 4 tests :

```
AddRowInvalidIndex() : void
AddRowBeginning() : void
AddRowMiddle() : void
AddRowEnd() : void
```

Si votre implémentation est fonctionnelle, elle devrait passer les 20 tests unitaires fournis.

Etude de complexité

Calculez la complexité temporelle pour chaque opération de **votre** implémentation de la classe `Matrix` (avec vos choix de collections génériques), en fonction du nombre de lignes m et de colonnes n. On n'étudiera que les complexités des opérations imposées.

Justifiez vos résultats avec des calculs, et éventuellement des illustrations de ce qui se passe en mémoire.

Résumez vos résultats dans un tableau de la forme suivante, avec des sous-cas en fonction des indices i et j si nécessaire (si le résultat est indépendant de i ou j, ceci devra être justifié également). Utilisez la notation de Landau (« grand O »).

Opération	Complexité
AddRow(i)	
AddColumn(i)	
RemoveRow(i)	
RemoveColumn(j)	
GetValue(i, j)	
SetValue(i, j)	

3 – Type Graph

Modélisation et stockage des sommets

Vous allez devoir réfléchir simultanément à la manière dont vous allez représenter et stocker les sommets du graphe. En effet, vous devez trouver les choix de conception permettant de minimiser la complexité temporelle des opérations suivantes :

- Accès à la valeur d'un sommet par son nom
- Accès au poids d'un arc par le nom de ses extrémités

Comme précédemment, a complexité égale, privilégiez la collection qui minimise la complexité des autres opérations (ajout/suppression de sommets) puis la facilité de mise en œuvre.

Complétez le diagramme de classes initié dans la partie précédente en ajoutant toutes les classes liées aux sommets et leurs associations, notamment avec Matrix.

Implémentation en C#

Complétez la classe `Graph` fournie pour que chaque méthode ait le comportement décrit dans le commentaire.

Attention : le graphe n'a pas le même comportement selon qu'il soit orienté ou non, comme illustré ci-dessous pour quelques opérations.

Graphe orienté	Oui	Non
Graphe après ajout de l'arc (A, B) avec un poids 42		
Poids de l'arc (A, B)	42	42
Poids de l'arc (B, A)	Erreur : arc inexistant	42
Ajout de l'arc (B, A) possible ?	Oui	Non
A est voisin de B	Non	Oui
B est voisin de A	Oui	Oui

Ajoutez tous les attributs/méthodes nécessaires, ainsi que les classes identifiées dans la phase de modélisation.

Ajustez le code si nécessaire, jusqu'à ce que tous les tests unitaires de la campagne `GraphTests` de passent.

Etude de complexité

Calculez la complexité des opérations de **votre** implémentation de la classe `Graph` (compte tenu des complexités du type `Matrix`, de votre modélisation et de votre stockage des sommets), en fonction du nombre de sommets n et d'arcs m.

Attention : pensez à bien prendre en compte l'ensemble des opérations réalisées sur les sommets et la matrice d'adjacence. Par exemple :

- La récupération du poids d'un arc à partir des noms des sommets nécessite de d'abord trouver l'indice de chaque sommet dans la matrice d'adjacence
- L'ajout ou la suppression d'un sommet affecte aussi la matrice d'adjacence.

Justifiez chaque résultat par des calculs, et des illustrations si nécessaire.

Résumez vos résultats dans un tableau de la forme suivante, avec la notation grand de Landau et des sous-cas si-nécessaire.

Opération	Complexité
AddVertex(name, value)	
RemoveVertex(name)	
GetVertexValue(name)	
SetVertexValue(name)	
GetNeighbors(vertexName)	
AddEdge(sourceName, destinationName)	
RemoveEdge(sourceName, destinationName)	
GetEdgeWeight(i, j)	
SetEdgeWeight(i, j, w)	

4 – Rendu

Soumissions régulières : sur GitHub

Soumettez régulièrement votre travail sur le dépôt Git.

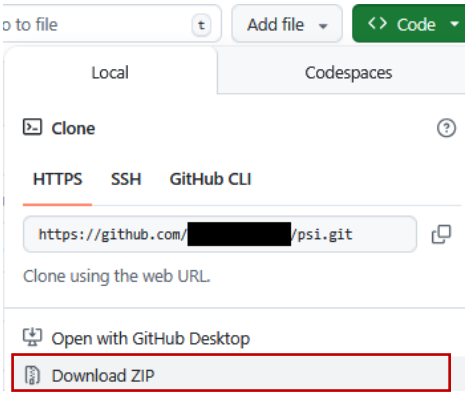
Tous les membres doivent contribuer au dépôt. Les commits de chaque membre permettront notamment à votre encadrant d'apprécier votre apport au projet.

Dépôt final pour évaluation : zip sur DVL

Le travail évalué sera votre dépôt sur DVL.

Pour rendre votre travail sur DVL :

1. Allez sur la page GitHub de votre dépôt
2. Cliquez sur « Code » puis « Download zip »



3. Renommez votre fichier zip en psi_objectif1_groupeX.zip, où X est votre nom de groupe (attribué au début du module). Par exemple, psi_objectif1_groupeNA1.zip pour Nantes, groupe de TD A, groupe de projet 1.
4. Déposez votre fichier zip sur DVL, activité « Rendu objectif1 », en utilisant le compte du **représentant du groupe** (ne faire qu'un dépôt, à ce nom).

Date limite de rendu : mardi 10 mars 2026, 23:59 Pénalités de retard : 1 point par heure de dépassement de la deadline.

Conseil : n'attendez pas le dernier moment pour soumettre votre travail : vous pouvez faire autant de rendus que vous le souhaitez dans la plage de dates autorisée.

5 – Evaluation

L'objectif 1 est uniquement évalué sur la réussite aux tests unitaires.

- Deux campagnes de tests : `MatrixTests` et `GraphTests`
- Pour chaque campagne : 1 point par test réussi ; 20 tests => note / 20
- Poids de chaque campagne : 50%
- Si le code ne compile pas ⇒ Note = 0

Vous connaissez donc votre note avant de rendre votre travail.

Rappel : certaines parties de l'objectif 1 seront évaluées lors du rendu du rapport (objectif 3) :

- Etude et choix des collections génériques
- Diagramme de classes UML illustrant les classes `Graph`, `Matrix`, et toutes les classes associées.
- Etude de complexité de votre implémentation du type `Graph` et `Matrix` (avec vos choix de collections).